

4조 Term Project 최종 보고서

2017124192 정찬우

2018124114 오상준

2021124057 김효정

프로그램 목표

영단어가 주어지고, 영단어에 포함된 철자를 맞추는 단어 맞추기 2인 경쟁 게임

<게임 룰>

- 단어에 포함된 철자를 맞추면 플레이어가 학교로 전진
- 틀린 철자를 입력하면 교수님이 학교로 전진

<승리 조건>

- 플레이어 본인이 상대 플레이어 보다 먼저 학교에 도착
- 상대 플레이어의 교수님이 먼저 학교에 도착

<패배 조건>

- 상대 플레이어가 플레이어 본인보다 먼저 학교에 도착
- 플레이어 본인의 교수님이 먼저 학교에 도착

주요 기능

▼ GUI 관련

1. 서버에게 전송할 철자를 버튼으로 입력
2. 현재 점수 현황을 보여주는 버튼
3. 서버로부터 수신한 데이터를 기반으로
 - Game Object들 렌더링
 - 현재 맞춰야 하는 단어에서 맞은 철자를 보여줌
4. 승리/패배 조건을 만족하면 결과를 이미지로 보여줌

▼ 네트워크 / 멀티 스레딩

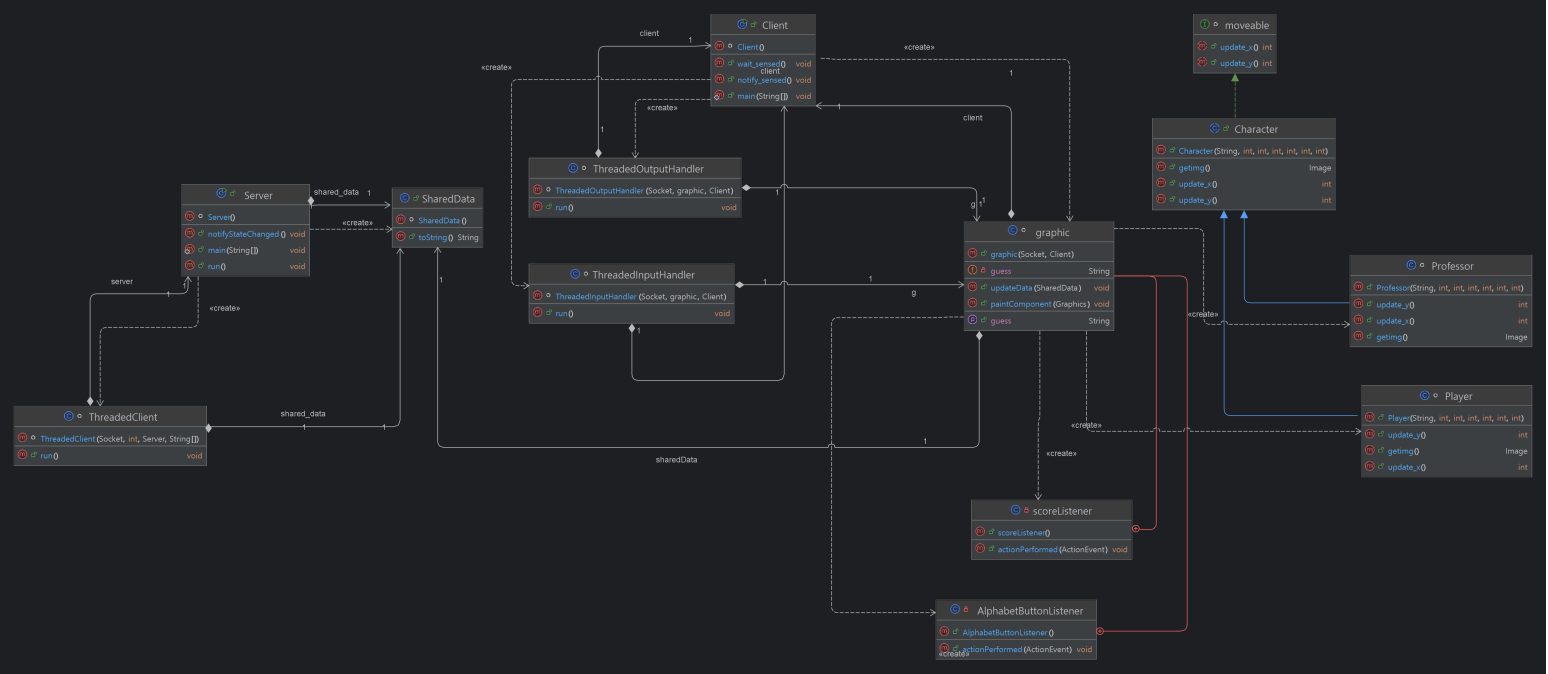
서버

1. 멀티 스레딩을 통해, 2개의 클라이언트로부터의 입력을 동시에 받는다.
2. 2개의 클라이언트로부터 char를 입력 받아서 두 클라이언트가 공유하는 데이터(**SharedData**)의 내용을 업데이트한다.
 - int 배열 업데이트 : 맞춰야 하는 단어에서, 맞은 철자가 1이고 아직 맞추지 못한 철자가 0인 배열
 - 현재까지 철자를 맞춘/틀린 횟수의 누적 합 갱신
3. 서버는 평소에는 대기(**wait()**)하고 있다가, 클라이언트의 입력이 들어오면 깨어나서(**notify()**) 클라이언트에게 **SharedData** 를 전송

클라이언트

1. 상대방의 입력 결과도 실시간으로 반영하기 위해, 서버로부터 데이터를 받는 input handler과 서버로 데이터를 보내는 output handler을 멀티 스레딩으로 병렬적으로 수행한다.
2. output handler는 평소에는 대기(**wait()**)하고 있다가, 사용자의 입력(**char**)이 들어오면 깨어나서(**notify()**) 서버에게 전송

전체 구조(UML)



- `Server` : 서버 프로그램, 클라이언트 연결 및 상태 변경 관리
- `ThreadedClient` : 클라이언트 스레드, 사용자 입력 처리
- `Client` : 클라이언트 프로그램, GUI 설정 및 통신 관리
- `ThreadedInputHandler` & `ThreadedOutputHandler` : 클라이언트 입력 및 출력 처리
- `graphic` : GUI 및 게임 상태 시각화
- `scoreListener` & `AlphabetButtonListener` : 각 버튼의 동작 구현
- `SharedData` : 게임 데이터 공유 객체

주요 소스 코드 설명

▼ Server.java

`Server` 클래스는 `state_changed` 변수를 lock으로 사용하여 wait하고 깨워진다. 깨워지면 두 클라이언트에게 `SharedData` 객체를 전송한다.

```
public class Server extends Thread
{
    public boolean state_changed;
    public SharedData shared_data;
    public ObjectOutputStream[] os;

    Server(){
        state_changed = false;
        shared_data = new SharedData();
        os = new ObjectOutputStream[2];
    }

    public synchronized void run() {
        while(true) {
            while(!state_changed) {
                try {
                    System.out.printf("%-10s Server is going to sleep, waiting for
state chage...\n", "[Server]:");
                    wait();
                    System.out.printf("%-10s Server has been awoken and goingto send!\n",
"[Server]:");
                } catch (InterruptedException e) {e.printStackTrace(System.out);}
            }

            try {
                for(int i=0; i<2; i++) {
                    shared_data.whoami = i;
                    os[i].writeObject(shared_data);
                    os[i].reset();
                    System.out.printf("%-10s Server has sent to Client %d\n", "[Server]:", i);
                }
                state_changed = false;
            } catch(IOException e) {e.printStackTrace(System.out);}

        }
    }

    public synchronized void notifyStateChanged() {
        state_changed = true;
        notify();
    }

    public static void main(...) {...}
}
```

`main()` 에서는 다음과 같은 일을 한다.

1. 각 클라이언트가 맞춰야할 단어들이 저장된 txt파일 읽기
2. 2개의 클라이언트로부터 요청을 받아들여 소켓 생성

`Server` 에게 두 소켓의 outputstream 전달

3. `accpet()` 한 각 `Socket` 을 이용하여 `ThreadedClient` 스레드와 `Server` 스레드 시작

```
public static void main(String[] args) {
    String fileName = "words.txt";

    // 각 클라이언트마다 단어 5개로 이루어진 String배열이 할당됨
    String[][] lists = new String[2][5];

    // 맞춰야하는 단어들이 저장된 txt파일 읽기
    try {
        BufferedReader br = new BufferedReader(new FileReader(fileName));

        for(int P1P2 = 0; P1P2<2; P1P2++) {
            br.readLine();
        }
    }
}
```

```

        for (int i = 0; i < 5; i++) {
            lists[P1P2][i] = br.readLine();
        }
    }

    br.close();
} catch (FileNotFoundException e) {e.printStackTrace();}
} catch (IOException e) {e.printStackTrace();}

Server server = new Server();

try {
    ServerSocket s = new ServerSocket(8189);
    System.out.printf("%-10s Server Socket Generated...\n", "[main]:");

    int P1P2 = 0;
    while(P1P2 < 2) { // 2명의 클라이언트가 만들어지는 것을 계속 감지
        Socket incoming = s.accept();
        System.out.printf("%-10s client%d accepted\n", "[main]:", P1P2);

        // incoming(소켓) : inputStream을 위해
        // P1P2(int) : Player 1/2 식별
        // server(Server) : 스레드끼리 Server객체를 공유하여, 어느 스레드든지
        //                     input이 들어오면 Server객체를 깨워서 두 클라이언트에
        //                     바뀐 내용 전송
        // list[P1P2](String[]) : 각 클라이언트가 맞춰야하는 단어들
        new ThreadedClient(incoming, P1P2, server, lists[P1P2]).start();

        System.out.printf("%-10s Thread%d starts\n", "[main]:", P1P2);

        // Server객체가 전송하기 위해 각 소켓의outputstream 전달
        server.os[P1P2] = new ObjectOutputStream(incoming.getOutputStream());
        P1P2++;
    }

    // 2개의 클라이언트가 연결되면 Server객체 시작
    server.start();
    System.out.printf("%-10s Server starts...\n", "[main]:");
} catch (Exception e) {e.printStackTrace(System.out);}
}

```

ThreadedClient 스레드는 소켓의 inputStream으로부터 철자를 읽고, 클라이언트로 보낼 **SharedData** 의 내용을 수정한다. 다음의 내용을 수정한다.

1. 입력된 character가 단어에 포함되는지 확인하여, 현재까지 맞춘 철자의 위치가 1이고 아직 맞추지못한 위치가 0인 int배열 **correct_character_index** 을 수정
2. 현재까지 맞춘 철자의 개수와 틀린 개수의 누적합 갱신 (**sum**)

SharedData 의 내용을 다 수정했으면 **server.notifyStateChanged()** 를 통해 **Server** 객체가 클라이언트로 전송할 수 있게 깨운다.

```

class ThreadedClient extends Thread
{
    private Server serer;
    private Socket incoming;

    private int id;                // 0:Player1, 1:Player2
    private String[] target_strings; // 맞춰야하는 단어들
    private int string_idx;        // target_strings를 인덱싱하기위한 int
    private SharedData shared_data;
    int[] correct_character_index;  // 현재 맞춘 철자들의 위치가 1인 int 배열
    String old_string;             // 단어가 바뀌는지 확인하기 위해

                                    // 단어가 바뀌면 correct_character_index를
                                    // 초기화해야한다.

    ThreadedClient(Socket i, int P1P2, Server s, String[] a){
        incoming = i;
        server = s;
        id = P1P2;
        shared_data = s.shared_data;
        target_strings = a;
        string_idx = 0;
    }

    public synchronized void run() {
        try {
            String answer = target_strings[string_idx]; // 맞춰야하는 정답 단어
            shared_data.current_target_word[id] = answer;
            old_string = "";

            // 처음에는 input이 없어도 client에게 정답 단어 전송
            server.notifyStateChanged();

            char guess; // 클라이언트로부터 입력 저장
            BufferedReader in = new BufferedReader

```

```

        (new InputStreamReader(incoming.getInputStream()));

while(true) { // 클라이언트로부터 데이터가 들어오는 것을 계속감지

    answer = target_strings[string_idx];
    shared_data.current_target_word[id] = answer;
    System.out.printf(
        "%-10s Player %d should guess : %s\n", "[Thread]: ", id, answer);

    // 단어가 달라지면 correct_character_index를 [0,...,0]으로 초기화
    if(old_string != answer) {
        shared_data.correct_character_index[id] = new int[answer.length()];
    }
    old_string = answer;

    // 클라이언트에서 char+'\n'을 보냄
    while ((guess = (char)in.read()) != '\n' ) {
        System.out.printf(
            "%-10s Socket %d received : \'%c\'\n", "[Thread]: ", id, guess);

        boolean is_correct = false;

        // 이전 상태의 correct_character_index 저장
        int[] old_correct_index = new int[answer.length()];
        for(int idx=0; idx < answer.length(); idx++) {
            old_correct_index[idx] = shared_data.correct_character_index[id][idx];
        }

        // 정답단어의 철자를 순회하면서 correct_character_index 갱신
        for(int idx=0; idx < answer.length(); idx++) {
            if(guess == answer.charAt(idx)) {
                is_correct = true;
                shared_data.correct_character_index[id][idx] = 1;
            }
        }

        if(!is_correct) { // 하나도 맞은 것이 없으면 틀린횟수 누적
            System.out.printf("%-10s Client %d Wrong\n", "[Tread]:", id);
            shared_data.sum[id][1]++;
        }
        else {
            System.out.printf("%-10s Client %d correct\n", "[Tread]:", id);
            System.out.printf(
                "%-10s\n\told:\n\t%s\n\tnew:\n\t%s\n", "[Thread]:",
                Arrays.toString(old_correct_index),
                Arrays.toString(correct_character_index));
            int current_increment = 0;

            // 0->1로 바뀐 것만 정답횟수에 누적
            for(int idx=0; idx < answer.length(); idx++) {
                if(shared_data.correct_character_index[id][idx] == 1) {
                    if(shared_data.correct_character_index[id][idx]
                        != old_correct_index[idx]) {
                        current_increment++;
                    }
                }
            }
            shared_data.sum[id][0] += current_increment;
        }

        // 전부 1이면 정답단어를 바꾼다.
        boolean is_allright = true;
        for(int idx=0; idx<answer.length();idx++) {
            if(shared_data.correct_character_index[id][idx] == 0)
                is_allright = false;
        }
        if(is_allright) {
            string_idx++;
            string_idx = (string_idx >= target_strings.length) ? 0 : string_idx;
        }
        server.notifyStateChanged(); // Server 객체를 깨워서 전송
    }
}
} catch (Exception e) {e.printStackTrace(System.out);}
}
}

```

▼ SharedData.java

서버 - 클라이언트는 소켓을 통해 `SharedData` 객체를 주고 받으며 통신한다.

`SharedData` 클래스의 멤버변수는 아래와 같다.

```
import java.io.Serializable;
import java.util.Arrays;

public class SharedData implements Serializable
{
    // index 0: for player1
    // index 1: for player2
    public static final int P1 = 0;
    public static final int P2 = 1;
    public int whoami; //플레이어 1, 플레이어2를 whoami를 통해 구분
    String[] current_target_word; //플레이어1, 2가 현재 맞추고 있는 단어
    public int[][] correct_character_index;
    public int[][] sum;

    ...

}
```

`correct_character_index` 는 플레이어가 현재 맞추고 있는 단어 안에서 어떤 철자들을 맞추었는지에 대한 정보를 담고 있다.

`correct_character_index` 는 2차원 배열로, `correct_character_index[0]` 은 Player 0이 어떤 철자를 맞추었는지에 대한 정보를 담고 있다. 예를 들어, Player 0이 단어 'school'에서 'h'와 'l'을 맞추었다면, `correct_character_index[0] = [0, 0, 1, 0, 0, 1]` 의 요소로 구성된다.

`sum` 은 player 2명이 틀린 답을 입력한 총 횟수를 저장한 2차원 배열이다.

`sum[0] = {player1이 맞춘 횟수, player1이 틀린 횟수}`

`sum[1] = {player2이 맞춘 횟수, player2이 틀린 횟수}`

`SharedData` 의 생성자 함수는 아래와 같다.

```
SharedData()
{
    correct_character_index = new int[2][];
    current_target_word = new String[2];
    sum = new int[2][2]; // [P1_correct, P1_wrong]
    sum[P1] = new int[] {0,0}; //게임 시작 전 player1의 correct/wrong count 초기화
    sum[P2] = new int[] {0,0}; ///게임 시작 전 player2의 correct/wrong count 초기화
}
```

▼ Client

`Client` 의 `main()` 에서는 `JFrame` 을 만들고 네트워크 통신을 위한 `Socket` 객체, `Client` 객체, GUI를 위한 `graphic` 객체, 서버로 부터 정보를 받기 위한 스레드를 담당하는 `ThreadedInputHandler` 와 서버로 정보를 보내기 위한 스레드를 담당하는 `ThreadedOutputHandler` 의 객체를 생성하고 스레드를 시작시킨다.

```
public static void main(String[] args)
{
    try
    {
        Client client = new Client();
        Socket clientsocket = new Socket("localhost", 8189);

        JFrame frame = new JFrame("Term Project");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        graphic g = new graphic(client, frame);

        ThreadedInputHandler t = new ThreadedInputHandler(clientsocket, g, client);
        t.start();

        //InputHandler & OutputHandler 스레드 시작
        new ThreadedOutputHandler(clientsocket, g, client).start();

        frame.getContentPane().add(g);
        frame.pack();
        frame.setVisible(true);

    } catch (UnknownHostException uhe) {
        uhe.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}
```

`Client` 클래스는 사용자로부터 정보가 들어오기를 감지한다. 이를 위한 메소드 2개가 구현 되어있다.

```
public class Client {
    public boolean char_sensed; //사용자로부터 입력을 감지하기 위한 변수

    Client(){
```

```

        char_sensed = false;
    }

    public synchronized void notify_sensed() {
        char_sensed = true;
        notify();
    }

    public synchronized void wait_sensed() {
        // char_sensed가 graphic객체에 의해 true가 될때 까지 wait()
        while(!char_sensed) {
            try {
                System.out.println("OutputHandler is going to sleep...");
                wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    public static void main(...) {...}
}

```

`wait_sensed` 메소드는 사용자로부터 정보가 들어오기 전까지 `Client` 스레드를 `wait()` 시킨다. `notify_sensed` 메소드는 호출되는 순간 `wait()` 중인 `Client` 스레드를 깨운다.

`ThreadedInputHandler` 와 `ThreadedOutputHandler` 는 둘 다 `Thread` 를 상속하는 class 이다.

```

class ThreadedInputHandler extends Thread
{
    ObjectInputStream is;
    graphic g;
    Client client;

    ThreadedInputHandler(Socket s, graphic g, Client client) {
        try {
            //객체를 정보로서 받기 위해 ObjectInputStream 사용
            this.is = new ObjectInputStream(s.getInputStream());
            this.g = g;
            this.client = client;
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public void run() {
        while (true) {
            try {
                //Server로부터 SharedData 객체가 정보로서 온다
                SharedData inputData = (SharedData) is.readObject();

                System.out.println(inputData);

                //사용자의 GUI에 정보를 반영시키기 위해 updateData() 사용
                g.updateData(inputData);

                //정보를 읽어올때 마다 paintComponent 호출
                g.repaint();
            } catch (ClassNotFoundException | IOException e) {
                e.printStackTrace();
            }
        }
    }
}

```

`ThreadedInputHandler` 에서는 통신을 위한 `Socket`, 렌더링을 위한 `graphic`, 동기화를 위한 `Client` 들의 객체를 인자로 받아 객체화 한다. `run()` 메소드 내부에서는 `Server` 에서 보내는 `SharedData` 를 읽어와 `graphic` 객체인 g에 업데이트하게 된다. 이는 GUI 화면 내 에서 `Server` 로부터 받은 데이터들을 가지고 게임을 진행하기 위함이다.

```

class ThreadedOutputHandler extends Thread {
    DataOutputStream os;
    graphic g;
    Client client;

    ThreadedOutputHandler(Socket s, graphic g, Client client) {
        try {
            this.os = new DataOutputStream(s.getOutputStream());
            this.client = client;
            this.g = g;
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

```

public void run() {
    while (true) {
        client.wait_sensed(); //사용자로부터 입력 받기 전까지 wait()

        System.out.println("OutputHandler wake up...");

        try {
            String guess = g.getGuess(); //사용자가 입력한 글자를 받아온다
            System.out.println("guess : " + guess);

            if (guess != null && !guess.isEmpty()) {
                os.writeBytes(guess + "\n"); //서버로 데이터 전송
            }
            client.char_sensed = false;
        } catch (IOException e) {e.printStackTrace();}
    }
}
}

```

`ThreadedOutputHandler` 도 `ThreadedInputHandler` 와 마찬가지로 `Socket`, `graphic`, `Client` 객체들을 인자로 받아온다. `run()` 메소드에서는 `Client` 쪽에서 글자 정보를 무한 반복하여 보내지 않기 하기 위해서, `wait()` 가 구현된 `Client` 내부의 메소드인, `wait_sensed()` 를 호출하고 `player`가 한 글자를 보내게 되면 그 후에 `notify()` 를 통해서 그 글자를 서버에 보낸 후 다시 `wait()` 를 시키게 된다.

`wait()` 와 `notify()` 는 같은 객체에 대해서만 작동한다. 우리의 경우 `wait`하는 것은 `ThreadedInputHandler` 이고, `notify`하는 것은 `graphic` 이기 때문에 `Client` 객체를 `ThreadedInputHandler` 와 `graphic` 에 공유시켜서, 간접적으로 `Client` 객체를 통해 동기화를 시켰다.

▼ graphic

`graphic` 클래스는 `Player`로 부터 글자를 입력 받기 위한 알파벳 버튼 26개 (배열로 선언) 와 점수판을 보여주기 위한 버튼, 그리고 서버로 부터 받아오는 데이터를 처리하기 위한 변수들이 선언 되어 있다.

```

private JButton[] alphabetButtons = new JButton[26]; //알파벳 버튼 26개
private String guess; //사용자가 입력한 단어
private JButton scorebutton; //점수판을 보여주는 버튼

private String word = " "; //Server로 부터 받아올 정답 단어

private int[] answer_score; //정답을 맞춘 index를 알기 위한 변수
private JLabel result_label; //게임의 결과를 보여주기 위해 라벨 생성
private JLabel scoredLabel1, scoredLabel2; //점수판 라벨1,2

//플레이어들이 각각 맞춘 갯수 합
private int total_count1 = 0;
private int total_count2 = 0;

//플레이어들이 각각 틀린 갯수 합
private int professor_count1 = 0;
private int professor_count2 = 0;

private SharedData sharedData;

Client client;
JFrame frame;

```

생성자 함수 내부에서는 라벨이나 버튼들을 추가하고 위치를 조정해준다. 버튼의 동작 구현을 위한 **Listener** 들도 추가한다.

```

this.client = client;
this.frame = frame;

setBackground(Color.darkGray);
setPreferredSize(new Dimension(900, 770));
setLayout(null);

result_label = new JLabel();
result_label.setBounds(350, 450, 200, 200);
add(result_label);

scorebutton = new JButton("Show Scores");
scorebutton.setBounds(90, 220, 150, 40);
add(scorebutton);

scoredLabel1 = new JLabel();
scoredLabel2 = new JLabel();

scoredLabel1.setBounds(30, 240, 500, 100);
scoredLabel1.setForeground(Color.white);

scoredLabel2.setBounds(30, 270, 500, 100);
scoredLabel2.setForeground(Color.white);

//점수판의 초기 설정은 보이지 않게 해놓음
scoredLabel1.setVisible(false);
scoredLabel2.setVisible(false);

```

```

add(scoredLabel1);
add(scoredLabel2);

//알파벳 26개의 해당 버튼들을 만들어주고 위치 조정 및 동작 구현을 위한
//Listener 추가
for (int i = 0; i < 9; i++)
{
    char letter = (char) ('a' + i);
    alphabetButtons[i] = new JButton(String.valueOf(letter));
    alphabetButtons[i].setBounds(205 + 55*i, 570, 45, 45);
    alphabetButtons[i].addActionListener(new AlphabetButtonListener());
    add(alphabetButtons[i]);
}

for (int i = 0; i < 10; i++)
{
    char letter = (char) ('j' + i);
    alphabetButtons[i] = new JButton(String.valueOf(letter));
    alphabetButtons[i].setBounds(178 + 55*i, 625, 45, 45);
    alphabetButtons[i].addActionListener(new AlphabetButtonListener());
    add(alphabetButtons[i]);
}

for (int i = 0; i < 7; i++)
{
    char letter = (char) ('t' + i);
    alphabetButtons[i] = new JButton(String.valueOf(letter));
    alphabetButtons[i].setBounds(258 + 55*i, 680, 45, 45);
    alphabetButtons[i].addActionListener(new AlphabetButtonListener());
    add(alphabetButtons[i]);
}

scorebutton.addActionListener(new scoreListener());

```

`updateData()` 메소드에서는 `graphic` 객체의 필드들에 **SharedData**로부터 받아온 데이터들을 넣어준다.

```

public void updateData(SharedData data) {
    //Server로 부터 받은 SharedData를 변수에 입력
    this.sharedData = data;
    this.word = data.current_target_word[data.whoami];
    this.total_count1 = data.sum[0][0];
    this.total_count2 = data.sum[1][0];
    this.professor_count1 = data.sum[0][1];
    this.professor_count2 = data.sum[1][1];
    this.answer_score = data.correct_character_index[data.whoami];

    //받은 데이터를 토대로 Frame의 제목을 설정해주어 플레이어 구분
    frame.setTitle("Term Project - Player " + (sharedData.whoami+1));
}

```

`paintComponent()` 메소드에서는 전반적으로 게임에서 쓰이는 이미지와 요소들의 위치를 조정해주고 각 버튼에 대한 **Listener**를 구현한다. 각 알파벳의 버튼들은 사용자가 누른 버튼의 글자를 인자로 받아서 서버로 보내는 동작을 하고 점수판을 보여주는 버튼은 점수 라벨을 보여주는 동작을 수행한다. 한 번 더 누르면 점수판을 숨겨주는 역할을 한다.

추가적으로 각 **Player**가 게임을 종료하면 각각의 이미지들을 띄우기 위한 조건 설정이 되어있고 점수판의 정보는 `SharedData` 의 데이터를 참조하여 각각의 라벨에 설정해준다.

```

public void paintComponent(Graphics page)
{
    super.paintComponent(page);

    page.setColor(Color.yellow);

    int c = 380;
    int x = (c/5)*4;
    int y = (c/5)*3;

    page.drawLine(450 - x, 300-y, 450, 300); //교수1
    page.drawLine(450 + x, 300-y, 450, 300); //교수2
    page.drawLine(450 - x, 300 +y, 450, 300); //player1
    page.drawLine(450 + x, 300+y, 450, 300); //player2

    page.setColor(Color.white);

    //Server로부터 받아온 정답 문자의 길이만큼 빈 줄 생성
    if (word != null)
    {
        int start_x = 450 - (50*(word.length()-20)/2);
        for (int i = 0; i < word.length(); i++)
        {
            page.drawLine(start_x + 50 * i, 540, (start_x+35) + 50 * i, 540);
        }
    }
}

```

```

//player가 맞춘 정답 표시
for (int i = 0; i < word.length(); i++)
{
    if (answer_score != null && answer_score[i] == 1)
    {
        page.drawString(String.valueOf(word.charAt(i)),
                        (start_x+13) + 50 * i, 530);
    }
}

//player와 professor, school의 이미지 생성
Professor pf1 = new Professor("pic/professor.png", 50, 40, 0,
                                professor_count1, 430 - x, 280-y);
Professor pf2 = new Professor("pic/professor.png", 50, 40, 1,
                                professor_count2, 420 + x, 280-y);

Player p1 = new Player("pic/mach.png", 50, 70, 0, total_count1,
                        430 - x, 260 +y);
Player p2 = new Player("pic/feet.png", 50, 120, 1, total_count2,
                        420 + x, 220+y);

ImageIcon Professor1 = new ImageIcon("pic/professor.png");
Image test = Professor1.getImage();

page.drawImage(pf1.getimg(), pf1.update_x(), pf1.update_y(),
    pf1.width, pf1.height, this);
page.drawImage(pf2.getimg(), pf2.update_x(), pf2.update_y(),
    pf2.width, pf2.height, this);
page.drawImage(p1.getimg(), p1.update_x(), p1.update_y(),
    p1.width, p1.height, this);
page.drawImage(p2.getimg(), p2.update_x(), p2.update_y(),
    p2.width, p2.height, this);

ImageIcon school = new ImageIcon("pic/school.png");
Image img = school.getImage();
page.drawImage(img, 410, 250, 80, 80, this);

//player의 승리 조건과 패배 조건 설정 및 각각의 이미지 생성
if(sharedData != null)
{
    if (sharedData.whoami == 0)
    {
        if (total_count1 >= 30 || professor_count2 >= 30)
        {
            ImageIcon result = new ImageIcon("pic/a.jpg");
            Image resultImage =
                result.getImage().getScaledInstance(200, 200, Image.SCALE_SMOOTH);
            result_label.setIcon(new ImageIcon(resultImage));
            System.out.println("win");
            new Timer(2000, e -> System.exit(0)).start(); // 2초 후 종료
        }
        else if (total_count2 >= 30 || professor_count1 >= 30)
        {
            ImageIcon result = new ImageIcon("pic/f.jpg");
            Image resultImage =
                result.getImage().getScaledInstance(200, 200, Image.SCALE_SMOOTH);
            result_label.setIcon(new ImageIcon(resultImage));
            System.out.println("lose");
            new Timer(2000, e -> System.exit(0)).start(); // 2초 후 종료
        }
    }

    else if (sharedData.whoami == 1)
    {
        if (total_count1 >= 30 || professor_count2 >= 30)
        {
            ImageIcon result = new ImageIcon("pic/f.jpg");
            Image resultImage =
                result.getImage().getScaledInstance(200, 200, Image.SCALE_SMOOTH);
            result_label.setIcon(new ImageIcon(resultImage));
            System.out.println("lose");
            new Timer(2000, e -> System.exit(0)).start(); // 2초 후 종료
        }

        else if (total_count2 >= 30 || professor_count1 >= 30)
        {
            ImageIcon result = new ImageIcon("pic/a.jpg");
            Image resultImage =
                result.getImage().getScaledInstance(200, 200, Image.SCALE_SMOOTH);
            result_label.setIcon(new ImageIcon(resultImage));
            System.out.println("win");
        }
    }
}

```

```

        new Timer(2000, e -> System.exit(0)).start(); // 2초 후 종료
    }
}

//점수판에 보여줄 정보 생성하여 라벨에 설정해줌
if (sharedData != null) {
    String scores1 = String.format("Player 1: Correct = %d, Wrong = %d,
        Finished left = %d \n",
        sharedData.sum[0][0], sharedData.sum[0][1], 30-sharedData.sum[0][0]);
    String scores2 = String.format("Player 2: Correct = %d, Wrong = %d,
        Finished left = %d",
        sharedData.sum[1][0], sharedData.sum[1][1],
        30-sharedData.sum[1][0]);
    scoredLabel1.setText(scores1);
    scoredLabel2.setText(scores2);

} else {
    scoredLabel1.setText("No data available");
    scoredLabel2.setText("No data available");
}
}

```

```

private class AlphabetButtonListener implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        JButton source = (JButton) event.getSource();
        guess = source.getText(); //사용자가 입력한 글자 받아오기
        client.notify_sensed(); //입력 시 wait() 하던 스레드를 깨워준다
    }
}

private class scoreListener implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        //점수판 버튼 클릭 시 마다 가시성 설정
        scoredLabel1.setVisible(!scoredLabel1.isVisible());
        scoredLabel2.setVisible(!scoredLabel2.isVisible());
    }
}

```

▼ Character.java

```

interface moveable
{
    public int update_x();
    public int update_y();
}

```

interface moveable 에서 각각의 클래스에서 필수로 구현해야 하는 메소드를 선언했다.

```

public abstract class Character implements moveable
{
    public int width;
    public int height;

    public int x;
    public int y;

    public int id;

    public ImageIcon imgicon;
    public Image img;

    public int count;

    public int start_x;
    public int start_y;

    public int current_x;
    public int current_y;

    // 이미지, 이미지 크기, 플레이어의 id, 총 이동해야 하는 거리, 좌표값을
    // 초기 인자로 받음
    public Character(String path,
                        int width, int height,
                        int id, int count,
                        int start_x, int start_y){

        this.width = width;
        this.height = height;
    }
}

```

```

        this.id = id;
        this.count = count;

        this.start_x = start_x;
        this.start_y = start_y;

        imgicon = new ImageIcon(path);
        img = imgicon.getImage();
    }

    public int update_x()
    {
        return current_x;
    }
    public int update_y()
    {
        return current_y;
    }

    public Image getimg()
    {
        return img;
    }
}

```

추상 클래스를 선언하여 각각의 Character(Player, Professor) 가 이동해야 하는 좌표값과 이미지생성에 대한 메소드를 구현하도록 설정한다.

```

class Professor extends Character
{
    public Professor(String path,
                     int width, int height,
                     int id, int count,
                     int start_x, int start_y)
    {
        super(path, width, height, id, count, start_x, start_y);
    }

    //graphic에서 좌표값 갱신을 원할때 updata_x(),update_y() 호출
    public int update_x()
    {
        if (id == 0)
        {
            current_x = start_x + 9*count;
        }

        else if ( id == 1)
        {
            current_x = start_x - 9*count;
        }

        return current_x;
    }

    public int update_y()
    {
        current_y = start_y + 7*count;
        return current_y;
        // y증가
        // x * (id 0:-1, 1: +1
    }

    public Image getimg()
    {
        return img;
    }
}

```

Professor가 이동해야 하는 좌표값을 설정해주고 이미지도 설정해준다.

```

class Player extends Character
{
    public Player(String path,
                  int width, int height,
                  int id, int count,
                  int start_x, int start_y)
    {
        super(path, width, height, id, count, start_x, start_y);
    }
}

```

```
//graphic에서 좌표값 갱신을 원할때 updata_x(),update_y() 호출
public int update_x()
{
    if (id == 0)
    {
        current_x = start_x + 9*count;
    }

    else if (id == 1)
    {
        current_x = start_x - 9*count;
    }

    return current_x;
}

public int update_y()
{
    current_y = start_y - 7*count;
    return current_y;
    // y증가
    // x * (id 0:-1, 1: +1
}

public Image getimg()
{
    return img;
}

}
```

Player가 이동해야 하는 좌표값을 설정해주고 이미지를 설정해준다.

객체지향 설계 기법의 적용

객체 설계

서버, 클라이언트 각각에서 송신, 수신을 담당하는 handler 클래스를 설계했다.

그래픽을 렌더링하는 클래스를 설계했다.

SharedData : 클라이언트 - 서버가 주고받는 클래스를 만들었다. 클라이언트의 그래픽을 렌더링할 때 필요한 내용으로 구성되어있다.

캡슐화

graphic 클래스의 캡슐화

- GUI렌더링에 사용되는 변수들인 graphic 객체의 필드들을 `private` 로 설정하여 다른 객체에서 수정할 수 없게 하였다.
ex) OutputHandler 객체가 graphic 객체의 private 멤버변수 guess에 접근할 때, graphic의 getGuess 메소드를 통해 guess에 접근한다.

상속

화면 내 움직이는 것들을 abstract class(`Character`) 를 상속하여 구현하였다. `Character` 를 상속받은 클래스들은 `paintComponent()` 시 호출되는 `drawImage()` 에 필요한 image, 좌표, width, height를 필드로 갖는다.

인터페이스

`Character` 클래스는 `movable` 인터페이스를 implement한다. `movable` 은 `Character` 클래스의 좌표값을 업데이트하는 `update()` 를 구현해야한다.

일 분담 및 공헌도

오상준, 김효정, 정찬우 33%